



Habib University
shaping futures

Habib University

OBJECT-ORIENTED PROGRAMMING FALL 2025

Week 04- Part 1

Introduction to Classes and Objects





THIS WEEK'S AGENDA

- 01. Dynamic Memory Allocation
- 02. Memory Leaks (Allocation on the Stack vs the Heap)
- 03. Dangling Pointers
- 04. Default Arguments to Functions
- 05. Function Overloading





DYNAMIC MEMORY ALLOCATION





DYNAMIC MEMORY ALLOCATION

- In C++ memory can be allocated dynamically.
- This memory is allocated on the heap rather than on the stack (which has memory for static arrays and other primitive data types).
- We can dynamically allocate memory on the heap in the following way:
 - `int * arr = new int[10];`
- Whenever memory is allocated dynamically using the new keyword, we need to explicitly deallocate it using the delete keyword, otherwise memory leaks could occur.
- It is also recommended that we set the pointer to nullptr after calling delete.





DYNAMIC MEMORY ALLOCATION

```
1  #include <iostream>
2
3  int main() {
4      // Dynamically allocate an integer array
5      int *arr = new int[5];
6      std::cout << "address of arr: " << arr << std::endl;
7
8      // Deallocate the memory
9      delete[] arr;
10     return 0;
11 }
```





MEMORY LEAKS





ALLOCATION ON STACK

- In most of the cases we have seen so far, variables (including all local ones) were being declared on the stack.
- These variables are deleted as soon as they go out of scope, hence causing no memory leaks.
- They are extremely fast and easy to allocate on stack.

```
1  #include <iostream>
2
3  int main() {
4      // Allocate an integer array on stack
5      int arr[5] = {1, 2, 3, 4, 5};
6      // You don't need to explicitly free stack memory
7      return 0;
8  }
```





MEMORY LEAKS

- The dynamically allocated memory on heap can cause memory leaks, since it retains itself even after the block of code that requested the allocation has ended.

```
1  #include <iostream>
2
3  int main() {
4      // Dynamically allocate an array of 5 doubles
5      double *dptr = new double[5]{5.3, 6.2, 7.4, 8.1, 9.0};
6
7      // Print the values in the array
8      for (int i = 0; i < 5; ++i) {
9          std::cout << dptr[i] << " ";
10     }
11     std::cout << std::endl;
12
13     // Return without deleting the allocated memory
14     return 0;
15 }
```





DANGLING POINTERS





DANGLING POINTERS

- A pointer pointing to a memory location which has been explicitly deleted (using the delete keyword) is known as a dangling pointer.
- This is because the pointer still holds the address of the memory location that has just been deleted.
- This issue can be resolved by setting the pointer variable to nullptr like so:
 - `delete[] arr;`
 - `arr = nullptr;`
- Another way to tackle this is to make sure we don't return pointers to a local variable from a function.





DANGLING POINTERS

```
1  #include <iostream>
2
3  int main() {
4      // Dynamically allocate an array of 5 doubles
5      double *dptr = new double[5]{5.3, 6.2, 7.4, 8.1, 9.0};
6
7      // Print the values in the array
8      for (int i = 0; i < 5; ++i) {
9          std::cout << dptr[i] << " ";
10     }
11     std::cout << std::endl;
12
13     // Deallocate the memory and set pointer to nullptr
14     delete[] dptr;
15     dptr = nullptr;
16
17     return 0;
18 }
```





DEFAULT PARAMETERS OF FUNCTIONS





DEFAULT ARGUMENTS

- A function can be called without specifying all its arguments, however its declaration must provide default values for those arguments that are not specified.
- Lets say you create a function for multiplying two integers, and you want the default to be multiplication by 1 if the second integer is not provided.
- The declaration for this function would thus be:
 - `int mult(int a, int b = 1);`
- This means that if the second integer is not passed to the function, the default value of 1 will be used.
- ***If one argument is missing when the function is called, it is assumed to be the last argument.***





DEFAULT ARGUMENTS

```
1  #include <iostream>
2
3  int mult(int a, int b);
4
5  int main() {
6      int x, y;
7      std::cout << "Enter first number: ";
8      std::cin >> x;
9      std::cout << "Enter second number: ";
10     std::cin >> y;
11     std::cout << "Product: " << mult(x, y) << std::endl;
12     std::cout << "Product with default second argument: " << mult(x) << std::endl;
13     return 0;
14 }
15
16 int mult(int a, int b = 1) {
17     return a * b;
18 }
```

Is this correct?





DEFAULT ARGUMENTS

```
1  #include <iostream>
2
3  int mult(int a, int b = 1);
4
5  int main() {
6      int x, y;
7      std::cout << "Enter first number: ";
8      std::cin >> x;
9      std::cout << "Enter second number: ";
10     std::cin >> y;
11     std::cout << "Product: " << mult(x, y) << std::endl;
12     std::cout << "Product with default second argument: " << mult(x) << std::endl;
13     return 0;
14 }
15
16 int mult(int a, int b) {
17     return a * b;
18 }
```

Is this correct?





FUNCTION OVERLOADING





FUNCTION OVERLOADING

- A function is said to be overloaded if it has multiple declarations of the same name but different signature.
- The signature of a function refers to the number, type and order of its parameters.
- The signature does not necessarily extend to the return type, therefore you could get a compiler error if the name and signature of the function is the same but there is a difference in return type.
 - `int func(int a, char b);`
 - `double func(int a, char b);`
 - **“error: functions that differ only in their return type cannot be overloaded”**





FUNCTION OVERLOADING

- There are three types of function overloading:
 - Overloading by number of parameters
 - Overloading by type of parameters
 - Overloading by order of parameters



OVERLOADING BY NUMBER OF PARAMETERS

- Two functions have the same name but different number of parameters.

```
1  #include <iostream>
2
3  int mult(int a, int b){
4      return a * b;
5  }
6  // Overloading by number of parameters
7  int mult(int a, int b, int c){
8      return a * b * c;
9  }
10
11 int main() {
12     std::cout << "mult(2, 3) = " << mult(2, 3) << std::endl;
13     std::cout << "mult(2, 3, 4) = " << mult(2, 3, 4) << std::endl;
14     return 0;
15 }
```

OVERLOADING BY TYPE OF PARAMETERS

- Two functions have the same name but different type of parameters.

```
1  #include <iostream>
2
3  int mult(int a, int b){
4      return a * b;
5  }
6  // Overloading by type of parameters
7  double mult(double a, double b){
8      return a * b;
9  }
10
11 int main() {
12     std::cout << "mult(2, 3) = " << mult(2, 3) << std::endl;
13     std::cout << "mult(2.5, 3.2) = " << mult(2.5, 3.5) << std::endl;
14     return 0;
15 }
```

OVERLOADING BY ORDER OF PARAMETERS

- Two functions have the same name but different order of parameters.

```
1  #include <iostream>
2
3  // Overloading by order of parameters
4  double mult(int a, double b){
5      return a * b;
6  }
7
8  double mult(double a, int b){
9      return a * b;
10 }
11
12 int main() {
13     std::cout << "mult(2, 3.4) = " << mult(2, 3.4) << std::endl;
14     std::cout << "mult(2.8, 2) = " << mult(2.8, 2) << std::endl;
15     return 0;
}
```